



Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

What is JavaScript?

From Wikipedia:

- ... high-level, dynamic, untyped, and interpreted programming language
- ... is prototype-based with first-class functions, ...
- ... supporting object-oriented, imperative, and functional programming
- ... has an API for working with text, arrays, dates and regular expressions

- Not particularly similar to Java: More like C crossed with Self/Scheme
 - C-like statements with everything objects, closures, garbage collection, etc.
- Also known as ECMAScript

Some thoughts about JavaScript

- Example of a scripting language
 - Interpreted, less declaring of things, just use them (popular today: e.g. python)
- Seems like it was designed in a rush
 - Some “Good Parts”, some not so good
 - Got a bad reputation
- Many programmers use a subset that avoids some common problems
- "use strict"; tweaks language to avoid some problematic parts
- Language being extended to enhance things: New ECMAScript every year!
 - Transpiling common so new features used: e.g ECMAScript Version N, TypeScript
- Code quality checkers (e.g. jslint, jshint, eslint) widely used

Good news if you know C - JavaScript is similar

```
i = 3;
i = i * 10 + 3 + (i / 10);
while (i >= 0) {
    sum += i*i; // Comment
    i--;
}
for (i = 0; i < 10; i++) {
}
/* this is a comment */
```

```
if (i < 3) {
    i = foobar(i);
} else {
    i = i * .02;
}
Most C operators work:
* / % + - ! >= <= > < && || ?:
function foobar(i) { return i;}
continue/break/return
```

Variable Declarations

Variables can be declared with `var` , `let` , or `const` .

Keyword	Scope	Re-assignable?	Preferred?
var	function	yes	no
let	block	yes	yes
const	block	no	yes

`let` and `const` were introduced in ES6. They are the preferred ways of declaring a variable.

Variable Declarations

Consider the following block of code...

```
let age = 27;  
const name = "Ashley";  
var hasCar = false;
```

Can we perform `age = 28` ? Yes! We use `let` .

Can we perform `name = "Carl"` ? No! We use `const` .

Is `hasCar` declared correctly? Yes, but we should use `let` instead of `var` .

JavaScript has dynamic typing

1. `var i;` `// Need to define variable ('use strict';), note: untyped`
2. `typeof i == 'undefined'` `// It does have a type of 'undefined'`
3. `i = 32;` `// Now: typeof i == typeof 32 == 'number'`
4. `i = "foobar";` `// Now: typeof i == typeof 'foobar' == 'string'`
5. `i = true;` `// Now typeof i == 'boolean'`

- Variables have the type of the last thing assigned to it
- Primitive types: undefined, number, string, boolean, function, object

Variable scoping with var: Lexical/static scoping

Two scopes: Global and function
local

```
var globalVar;           // Global variable

function foo() {
  var localVar;          // Function-scoped variable
  if (globalVar > 0) {
    var localVar2 = 2;    // Still function-scoped, not just inside the 'if'
  }
  // localVar2 is valid here
}
```

Variable scoping with var: Lexical/static scoping

All var statements hoisted to top of scope:

```
function foo() {  
    var x;  
    x = 2;  
}
```

```
function foo() {  
    x = 2;  
    var x;    // declaration is moved up  
}
```

Var scope problems

- Global variables are bad in browsers - Easy to get conflicts between modules
- Hoisting can cause confusion in local scopes (e.g. access before value set)

```
function() {  
  console.log('Val is:', val); // prints undefined (not an error!)  
  for (var i = 0; i < 10; i++) {  
    var val = "different string"; // declaration hoisted to top  
  }  
}
```

- Some JavaScript guides suggest always declaring all var at function start
- ES6 introduced non-hoisting, scoped let and const with explicit scopes
Some coding environments ban var and use let or const instead

number type

number type is stored in floating point (i.e. double in C)

$$\text{MAX_INT} = (2^{53} - 1) = 9007199254740991$$

Some oddities: NaN, Infinity are numbers

$$1/0 == \text{Infinity}$$

$$\text{Math.sqrt}(-1) == \text{NaN}$$

Watch out:

```
0.1 + 0.2 === 0.3    // false
0.1 + 0.2             // 0.30000000000000004
```

bitwise operators (e.g. $\sim$, $\&$, $|$, $\wedge$, $\gg$, $\ll$, $\ggg$) are 32bit!

string type

string type is variable length (no char type)

```
let foo = 'This is a test';  
let bar = "This is also a test";
```

```
foo.length // 14
```

+ is string concat operator

```
foo = foo + 'XXX';  
// Result: "This is a testXXX"
```

string type

JavaScript strings come with many built-in methods for manipulation:

- **Search/Indexing:**
 - `indexOf()` → Find position of a substring.
 - `charAt()` → Get a character at a position.
 - `match()`, `search()` → Regular expression operations.
- **Modify/Replace:**
 - `replace()` → Replace part of the string.
 - `slice()`, `substr()` → Extract parts of a string.
- **Case Conversion:**
 - `toUpperCase()` → Convert to uppercase.
 - `toLowerCase()` → Convert to lowercase.

```
'foo'.toUpperCase(); // "F00"
```

boolean type

- A boolean in JavaScript can only be true or false.
- Language classifies values as either truthy or falsy
 - Used when a value is converted to a boolean e.g. `if (foo) { ... }`
- Falsy values:
 - `false`
 - `0` (zero)
 - `NaN` (Not-a-Number)
 - `""` (empty string)
 - `undefined`
 - `null`

```
if (0) {  
    console.log("This will not run");  
}
```

boolean type

- A boolean in JavaScript can only be true or false.
- Language classifies values as either **truthy** or **falsy**
 - Used when a value is converted to a boolean e.g. `if (foo) { ... }`
- **Truthy:**

Everything not falsy is truthy. Examples include:

- Non-empty strings (`"hello"`)
- Any non-zero number (`42` , `-5`)
- Objects (`{}` , `[]`)
- Functions

```
if ("hello") {  
  console.log("This will run"); // truthy  
}
```

undefined and null

- undefined - does not have a value assign

let x; // x has a value of undefined

x = undefined; // It can be explicitly store

typeof x == 'undefined'

- null - a value that represents whatever the user wants it to

Use to return special condition (e.g. no value)

typeof null == 'object'

- Both are falsy but not equal (null == undefined; null !== undefined)

function type

```
function foobar(x) {  
  if (x <= 1) {  
    return 1;  
  }  
  return x * foobar(x - 1);  
}
```

`typeof foobar == 'function'; foobar.name == 'foobar'`

- Function definitions are hoisted (i.e. can use before definition)
- Can be called with a different number arguments than definition
 - Array arguments variable (e.g. `arguments[0]` is first argument)
 - Unspecified arguments have value `undefined`
- All functions return a value (default is `undefined`)

"First class" function example

Function Assigned to a Variable:

```
let aFuncVar = function (x) {  
  console.log('Func called with', x);  
  return x + 1;  
};
```

Passing a Function as an Argument:

```
myFunc(aFuncVar);
```

Function Receiving Another Function

```
function myFunc(routine) {  
  console.log('Called with', routine.toString());  
  let retVal = routine(10);  // routine is executed here with argument 10  
  console.log('retVal', retVal);  
  return retVal;  
}
```

object type

- Object is an unordered collection of **name-value** pairs called properties

```
let foo = {};    // empty object
let bar = { name: "Alice", age: 23, state: "California" };
```

- Name can be any string: `let x = { "": "empty", "---": "dashes" };`
- Referenced either like a structure or like a hash table with string keys:
 `bar.name` or `bar["name"]`
 `x["---"]` // have to use hash format for illegal names
 `foo.nonExistent == undefined`
- Global scope is an object in browser (i.e. `window[prop]`)

Properties can be added, removed, enumerated

- To add, just assign to the property:

```
let foo = {};  
foo.name = "Fred";  
console.log(foo.name); // "Fred"
```

- To remove use delete:

```
let foo = { name: "Fred" };  
delete foo.name;  
console.log(foo.name); // undefined
```

- To enumerate use Object.keys():

```
let bar = { name: "Alice", age: 23 };  
console.log(Object.keys(bar)); // ["name", "age"]
```

Arrays

```
let anArr = [1, 2, 3];  
typeof anArr; // "object"
```

Arrays are Objects

Indexed by non-negative integers:

```
console.log(anArr[0]); // 1  
console.log(anArr[2]); // 3
```

Can be sparse and polymorphic:

```
anArr[5] = "FooBar";  
console.log(anArr); // [1, 2, 3, <2 empty items>, "FooBar"]
```

Like strings, have many methods:

- `push()` → add element at the end
- `pop()` → remove last element
- `shift()` → remove first element
- `unshift()` → add element at the beginning
- `sort()`, `reverse()`, `splice()` → manipulate order and content

Arrays

Oddity: can store properties like objects (e.g. `anArr.name = 'Foo'`)

```
anArr.name = "Foo";  
console.log(anArr.name); // "Foo"
```

Setting `anArr.length = 0` empties the array:

```
anArr.length = 0;  
console.log(anArr); // []
```

Setting length to a larger value adds empty slots:

```
anArr.length = 5;  
console.log(anArr); // [empty × 5]
```

Dates

```
let date = new Date();
```

Are special objects: `typeof date; // "object"`

The number of milliseconds since midnight January 1, 1970 UTC

Timezone needed to convert. Not good for fixed dates (e.g. birthdays)

Many methods for returning and setting the data object. For example:

```
date.valueOf() = 1452359316314
```

```
date.toISOString() = '2016-01-09T17:08:36.314Z'
```

```
date.toLocaleString() = '1/9/2016, 9:08:36 AM'
```

Exceptions - try/catch

- Error reporting frequently done with exceptions

Example:

```
nonExistentFunction();
```

Terminates execution with error:

Uncaught ReferenceError: nonExistentFunction is not defined

- Exception go up stack: Catch exceptions with try/catch

```
try {  
  nonExistentFunction();  
} catch (err) {  
  // `err` is an Error object  
  console.log("Error call func", err.name, err.message);  
}
```

Exceptions - throw/finally

- Raise exceptions with throw statement

```
try {  
    throw "Help!";  
} catch (errstr) { // errstr === "Help!"  
    console.log('Got exception', errstr);  
} finally {  
    // This block is executed after try/catch  
}
```

- Conventions are to throw sub-classes of Error object

```
console.log("Got Error:", err.stack || err.message || err);
```

Regular Expressions

```
let re = /ab+c/;           // literal syntax
let re2 = new RegExp("ab+c"); // constructor syntax
```

Defines a pattern that can be searched for in a string

```
"abbc".match(/ab+c/); // Matches ["abbc"]
```

Methods That Use Regex:

- **String methods:**
 - `search()` → Finds index of match
 - `match()` → Returns the matches
 - `replace()` → Replaces matched text
 - `split()` → Splits a string based on the regex
- **RegExp methods:**
 - `exec()` → Executes a search and returns match details
 - `test()` → Returns `true / false` if the pattern exists

Regular Expressions

Literals & groups :

- `a b c` – literal characters
- `.` – any char (except newline unless `s`)
- `(abc)` – **capturing** group
- `(?:abc)` – **non-capturing** group
- `(?<name>abc)` – **named** capturing group

Special characters (metacharacters): Characters with special meaning, like `.` (any character), `*` (zero or more occurrences), `+` (one or more occurrences), `?` (zero or one occurrence), `^` (start of string/line), `$` (end of string/line).

https://www.w3schools.com/js/js_regexp.asp

Regular Expressions

Cool combination of CS Theory and Practice: CS143

Uses:

Searching: Does this string have a pattern I'm interested in?

```
/abc/.test("123abc456"); // true
```

Parsing: Interpret this string as a program and return its components

```
let parts = "2025-08-29".match(/(\d{4})-(\d{2})-(\d{2})/);  
// ["2025-08-29", "2025", "08", "29"]
```

Getting JavaScript into a web page

- By including a separate file:

```
<script type="text/javascript" src="code.js"></script>
```

- Inline in the HTML:

```
<script type="text/javascript">
```

```
//<![CDATA[
```

Javascript goes here...

```
//]]>
```

```
</script>
```

Next Class



Week 2:

Topic:

- Javascript